Chapter 8: Memory Management

Chapter 8: Memory Management

- Background
- Swapping
- Contiguous Allocation
- Paging
- Segmentation

Background

- It is a method of managing various types of memory especially the primary memory (RAM).
- Program must be brought secondary memory (from disk) into primary memory and placed within a process for it to be run.
- Main memory (RAM) and registers are only storage CPU can access directly.

Multiprogramming

- Multiple Programs stored at the same time
  - into different areas of memory
- Processor needs to context switch
- Programs use memory addresses / references
- Proper addressing is required
  - no matter where the program is located in memory

Degree of Multiprogramming

- CPU utilization = $1 - P^n$
  - n process in memory
  - P is the I/O block time by one process
  - P^n probability of all process are waiting for I/O

- I/O waiting 80% of time
  - CPU utilization = $1 - 0.8^1$ =20% [1 process @ memory]
  - CPU utilization = $1 - 0.8^3$ =49% [3 processes @ memory]
  - CPU utilization = $1 - 0.8^{10}$ =89% [10 processes @ memory]

Logical Address Space

- **Input queue** – collection of processes on the disk that are waiting to be brought into memory to run the program

- Each **process** has a **separate address space**
  - **Base register**: smallest legal physical memory address
  - **Limit register**: size of the range

- Memory Background
  - Stream of memory address.
  - They are used for instruction or data.

A base and a limit register define a logical address space

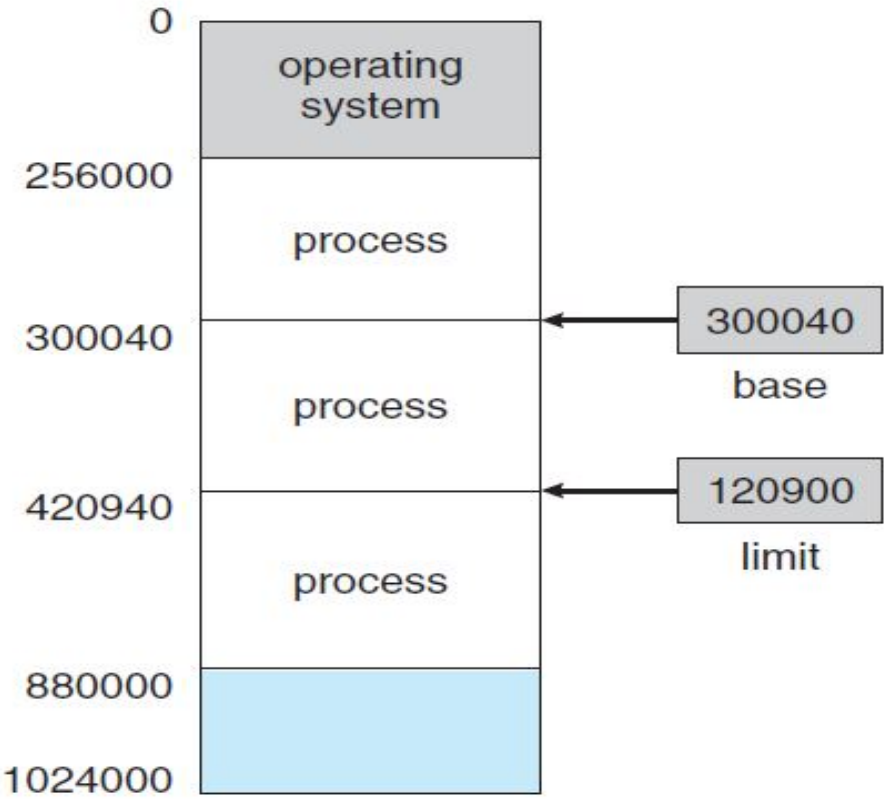


Figure 8.1 A base and a limit register define a logical address space.

If the base register holds 300040 and limit register 120900, the program can legally access all address from 300040 through 420940 (inclusive)

HW address protection with base and limit registers

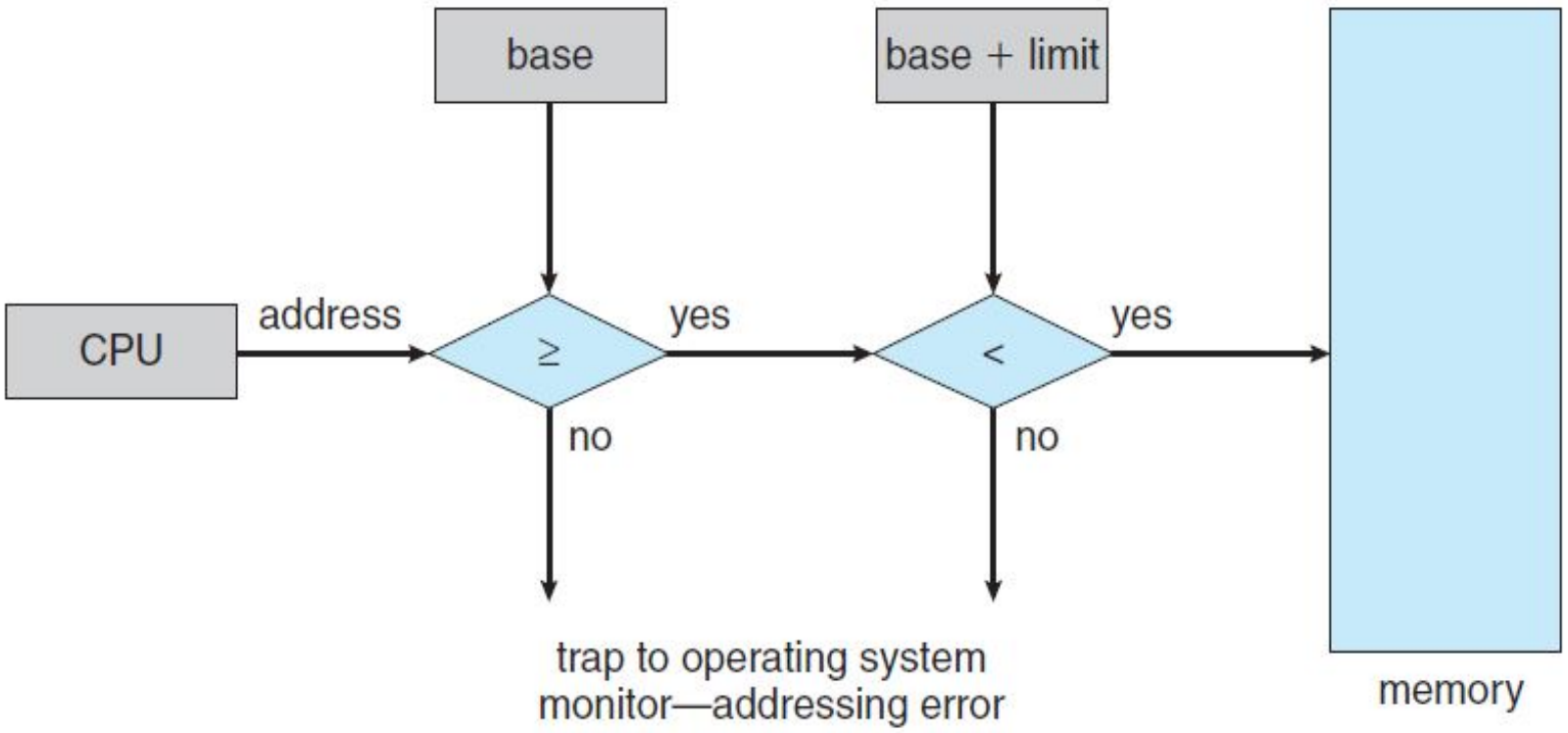


Figure 8.2 Hardware address protection with base and limit registers.

Binding of Instructions and Data to Memory

Address binding of instructions and data to memory addresses can happen at three different stages

- **Compile time**: If memory location known a priori, ***absolute code*** can be generated; must recompile code if starting location changes
- **Load time**: Must generate ***relocatable code*** if memory location is not known at compile time
- **Execution time**: Binding delayed until run time if the process can be moved during its execution from one memory segment to another. Need hardware support for address maps (e.g., *base* and *limit registers*).

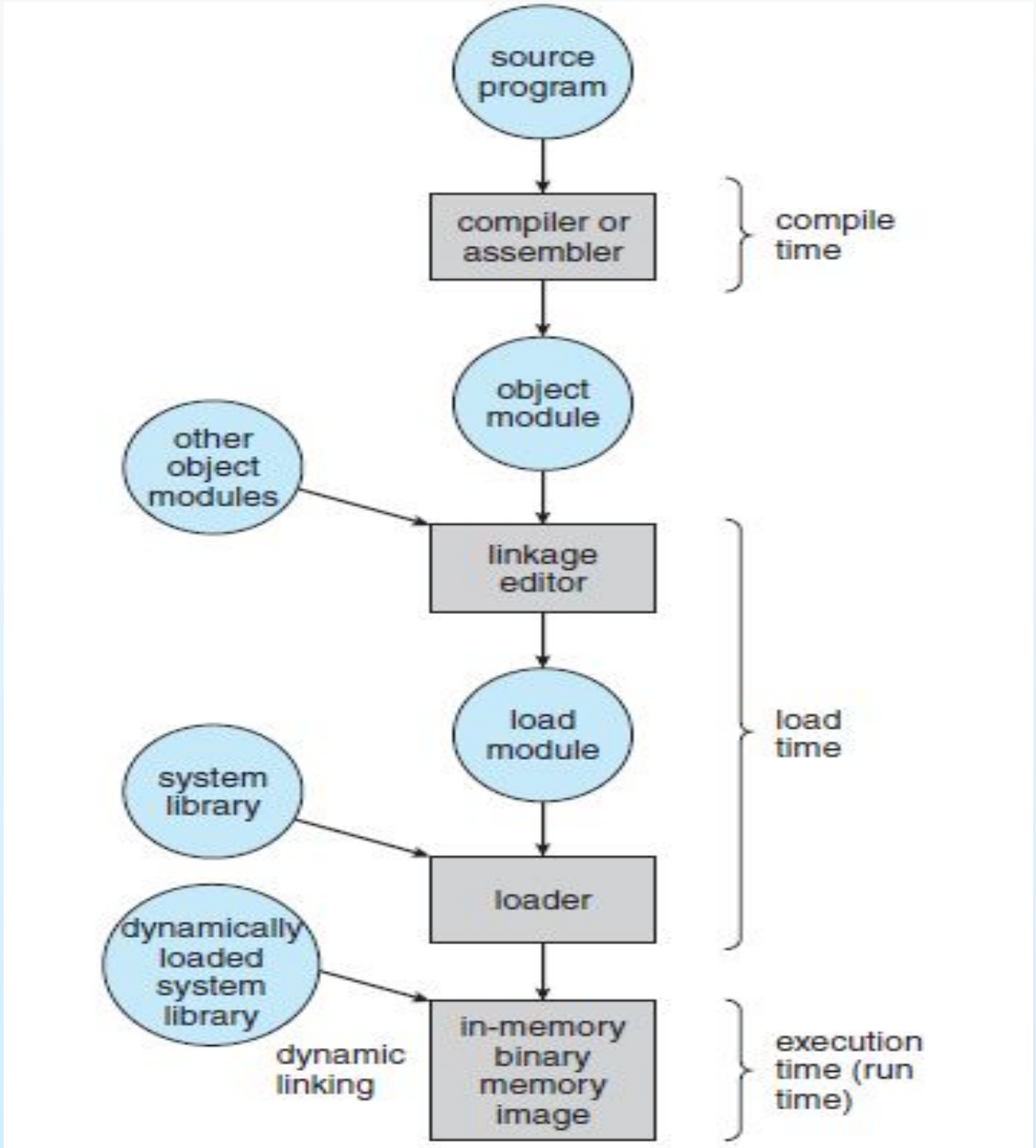


Figure 8.3 Multistep processing of a user program.

Logical vs. Physical Address Space

- The concept of a logical *address space* that is bound to a separate *physical address space* is central to proper memory management
  - **Logical address** – generated by the CPU; also referred to as *virtual address*
  - **Physical address** – address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes;
- logical (virtual) and physical addresses differ in execution-time address-binding scheme

Memory-Management Unit (MMU)

- Hardware device that **maps** virtual to physical address
- In MMU scheme, the value in the relocation register is added to every address generated by a user process at the time it is sent to memory
- The user program **deals with** *logical* addresses; it never sees the *real* physical addresses

Dynamic relocation using a relocation register

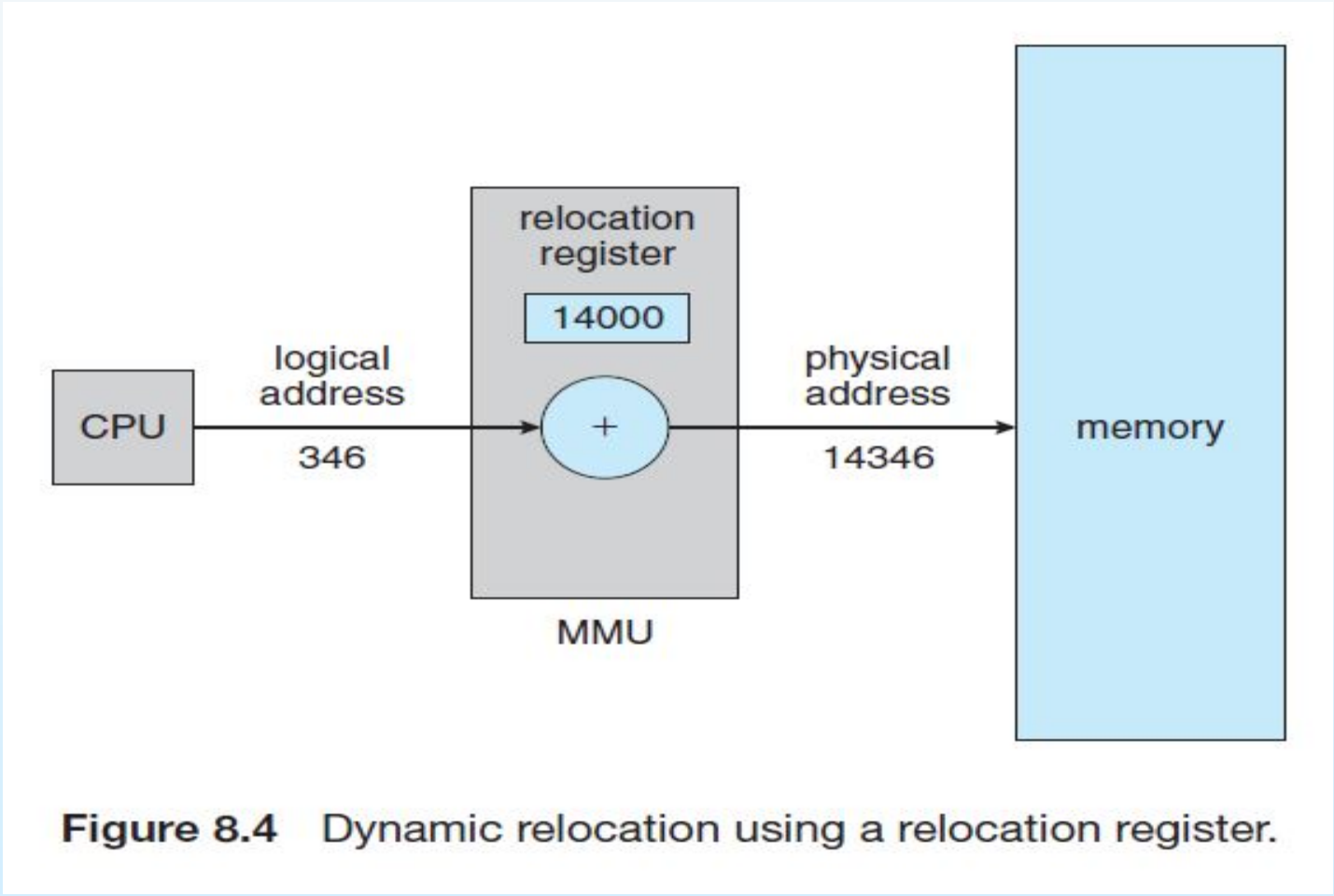


Figure 8.4 Dynamic relocation using a relocation register.

Swapping

- A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)

Schematic View of Swapping

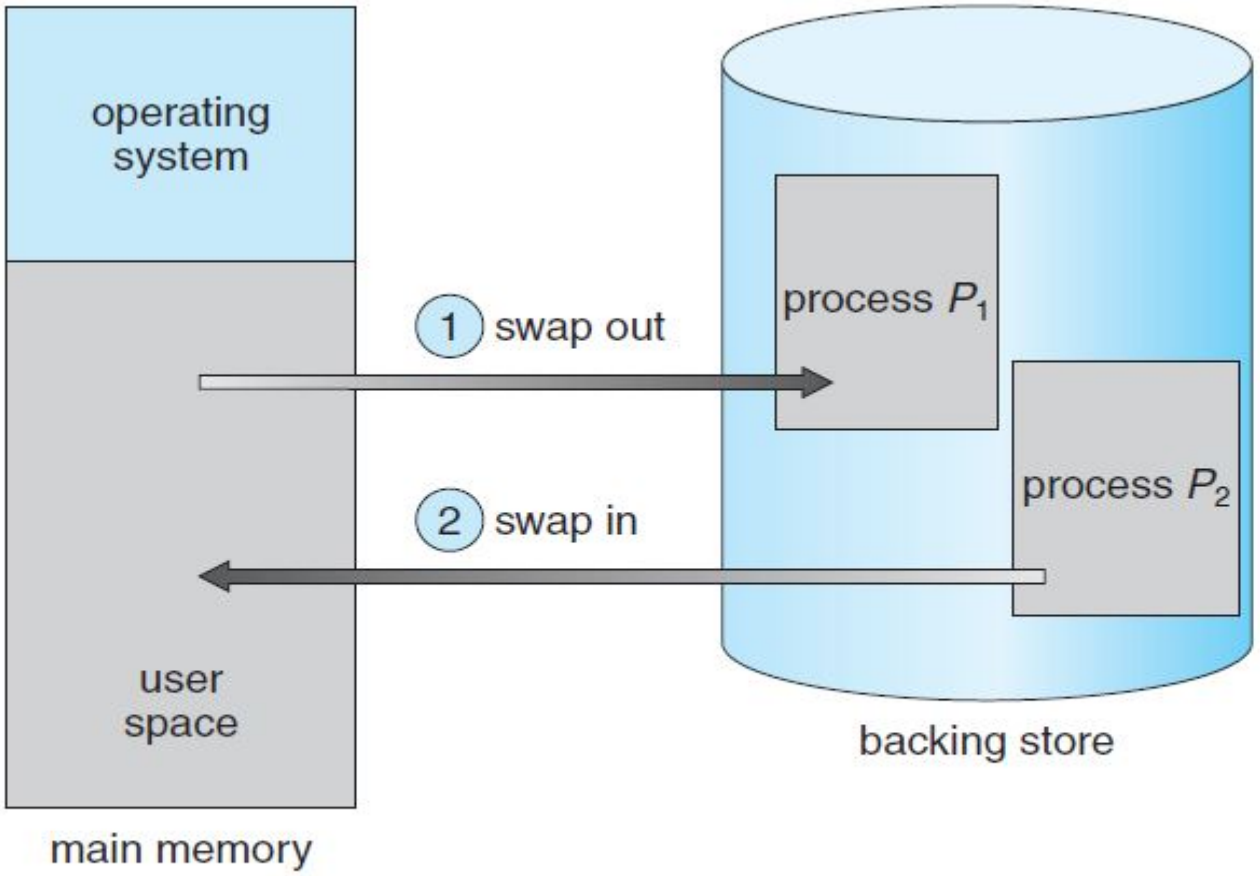


Figure 8.5 Swapping of two processes using a disk as a backing store.

Swapping

- The context-switch time in such a swapping system is fairly high. To get an idea of the context-switch time, let's assume that the user process is **100 MB** in size and the backing store is a standard hard disk with a transfer rate of **50 MB** per second. The actual transfer of the 100 MB process to or from main memory takes

  100 MB/50 MB per second = 2 seconds

- The swap time is 200 milliseconds. Since we must swap both out and in, the total swap time is about 400 milliseconds. (Here, we are ignoring other disk performance aspects, which we cover in Chapter 10.)

Swapping

- Swapping is constrained by other factors as well. If we want to swap a process, we must be sure that it is completely idle. Of particular concern is any pending I/O. A process may be waiting for an I/O operation when we want to swap that process to free up memory. However, if the I/O is asynchronously accessing the user memory for I/O buffers, then the process cannot be swapped. Assume that the I/O operation is queued because the device is busy. If we were to swap out process *P*1 and swap in process *P*2, the I/O operation might then attempt to use memory that now belongs to process *P*2.

- There are **two main solutions** to this problem: never swap a process with pending I/O, or execute I/O operations only into operating-system buffers.

- Transfers between operating-system buffers and process memory then occur only when the process is swapped in. Note that this **double buffering** itself adds overhead. We now need to copy the data again, from kernel memory to user memory, before the user process can access it.

Contiguous Allocation

- Memory mapping and protection
  - **Relocation-register** scheme used to protect user processes from each other, and from changing operating-system code and data
  - Relocation register contains value of smallest physical address
  - **Limit register** contains range of logical addresses – each logical address must be less than the limit register

Relocation and Limit registers

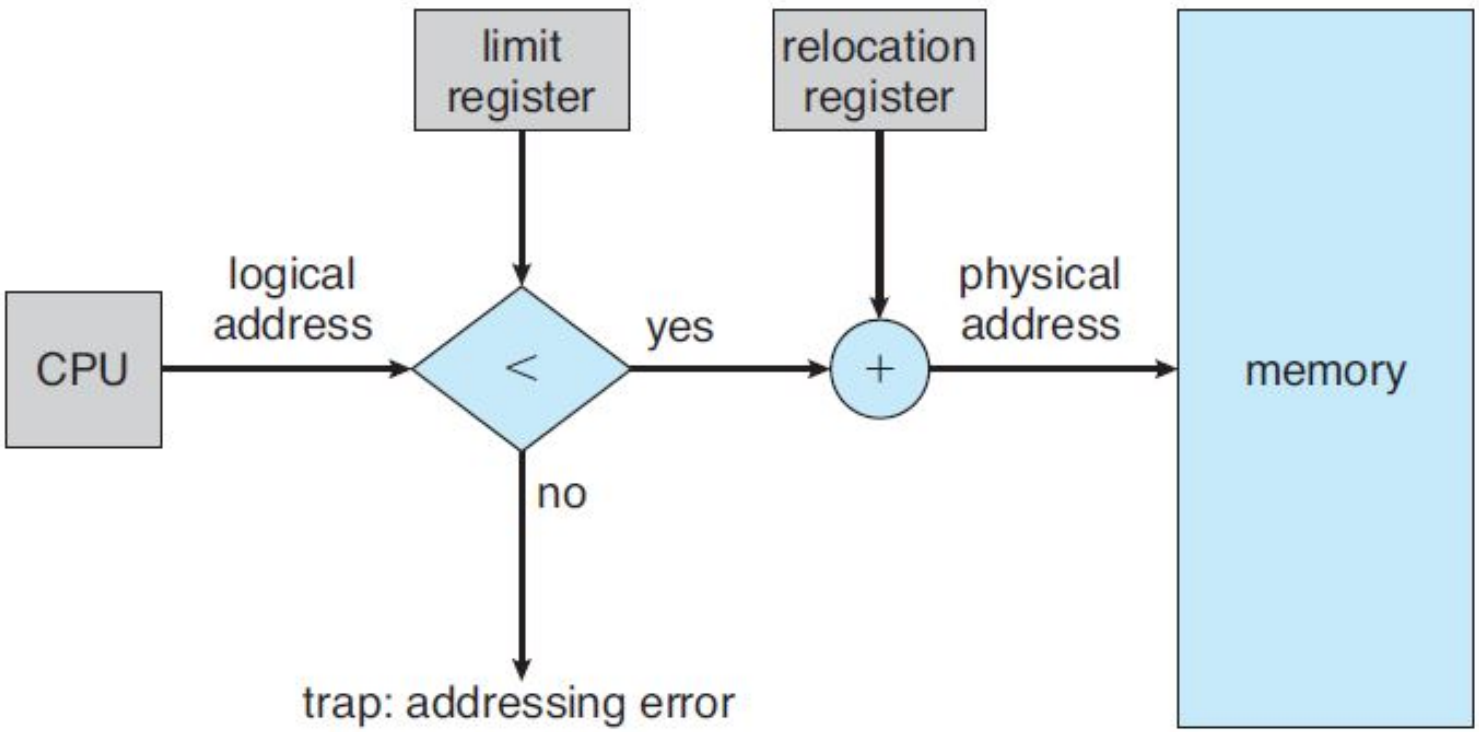
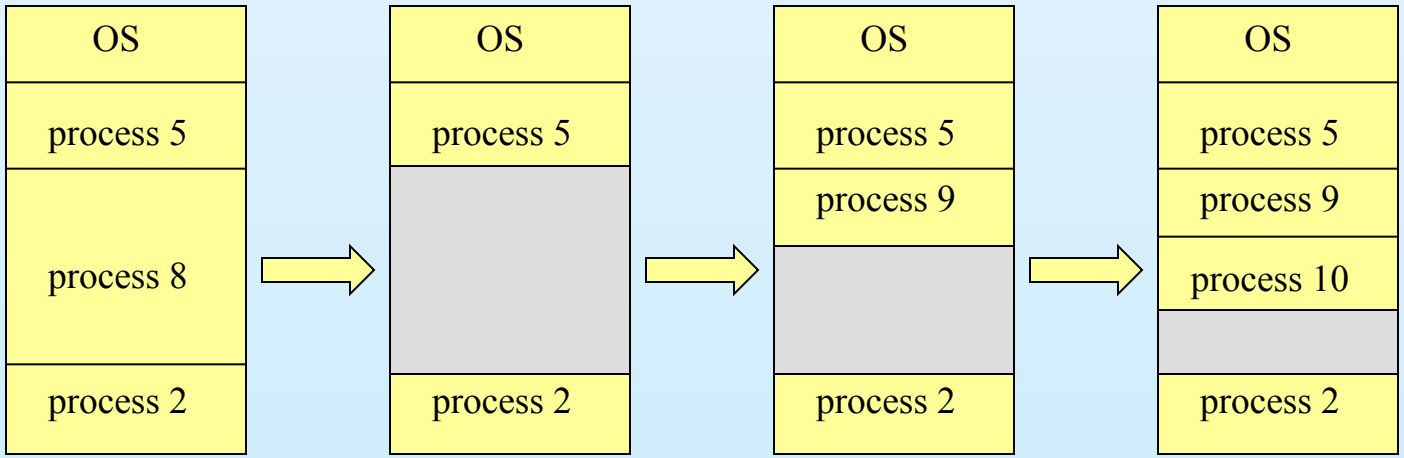


Figure 8.6 Hardware support for relocation and limit registers.

Contiguous Allocation (Cont.)

- Multiple-partition allocation
  - *Hole* – block of available memory; holes of various size are scattered throughout memory
  - When a process arrives, it is allocated memory from a hole large enough to accommodate it
  - Operating system maintains information about:
    a) allocated partitions b) free partitions (hole)



Contiguous Allocation (Cont.)

- **Fixed Partitioning**

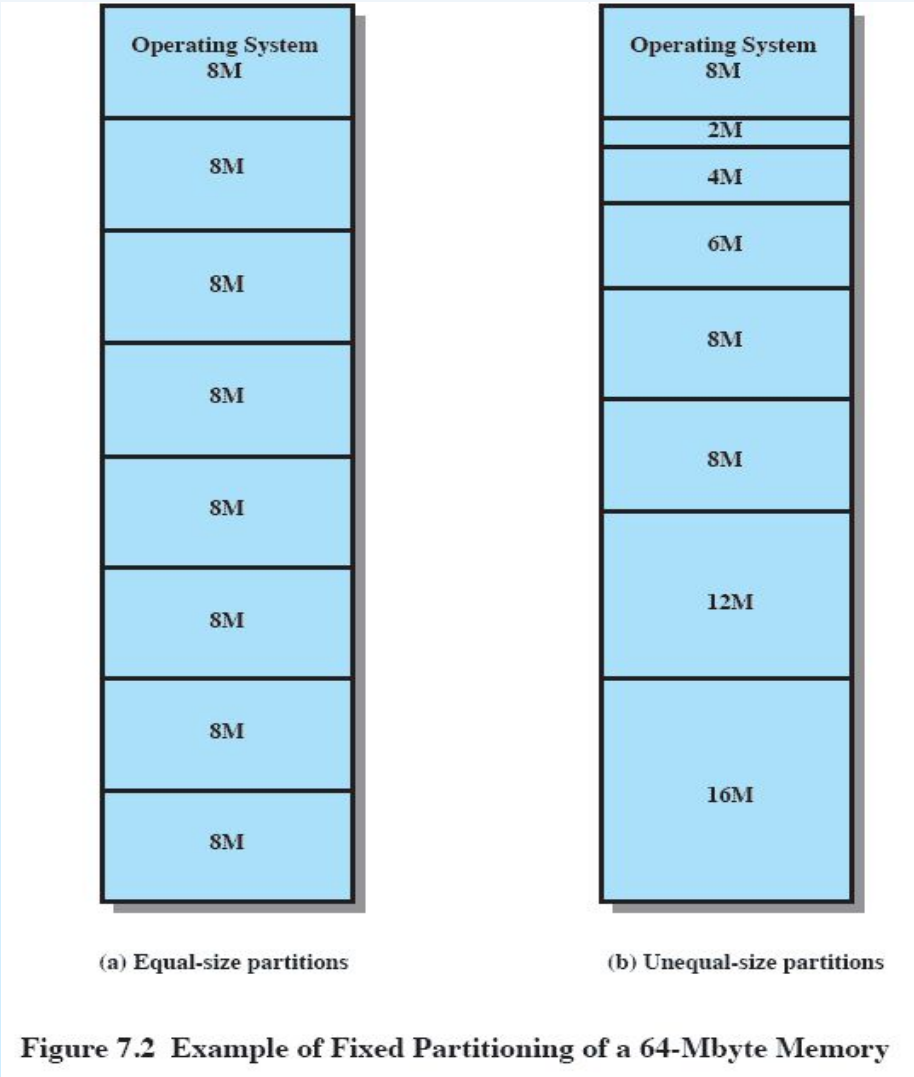


Figure 7.2 Example of Fixed Partitioning of a 64-Mbyte Memory

Fixed Partitioning

- Number of partitions are fixed
- Size of each partition may or may not be same
- Since contiguous allocation, so spanning is not allowed

Disadvantages:

- Internal fragmentation
- Process size is limited
- Limitations on degree of multiprogramming

- **Variable Partitioning**

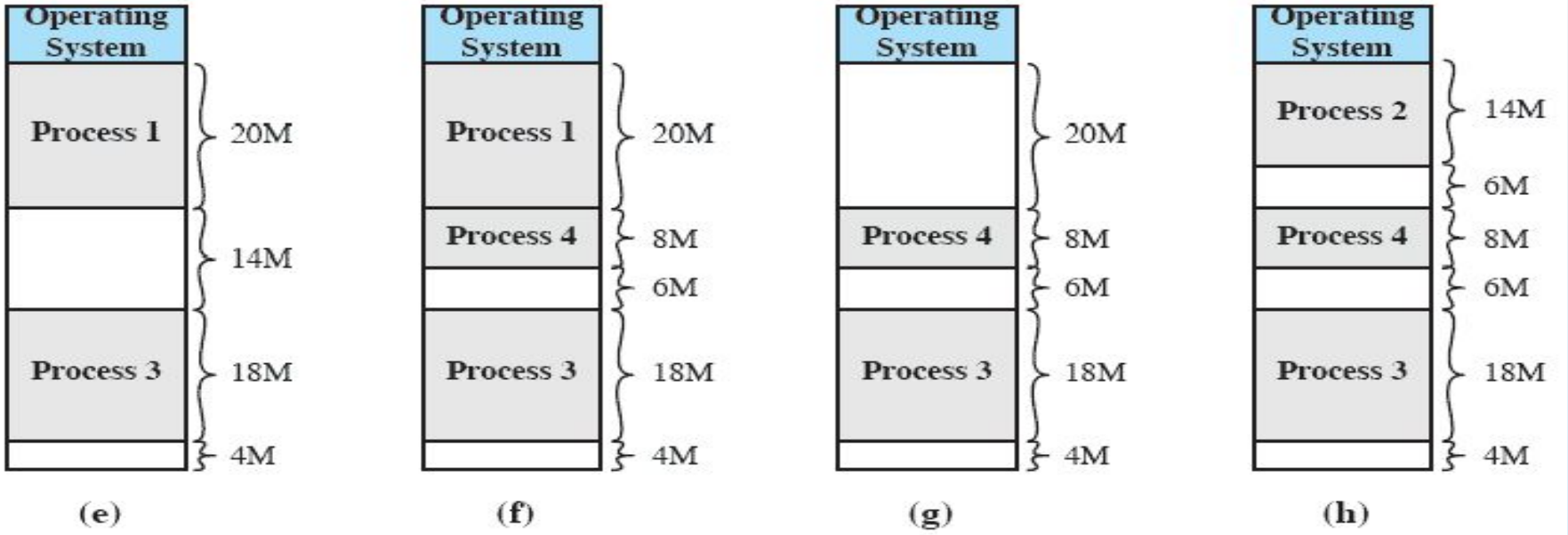


Figure 7.4 The Effect of Dynamic Partitioning

Variable Partitioning

- There are no partitions beforehand.
- Partition will be created according to the process size during runtime.

Advantages:

- No internal fragmentation
- No limitation on number of processes
- No limitation on process size

Disadvantages:

- External fragmentation (hole) will be created.
- Allocation and reallocation is complex.

Dynamic Storage-Allocation Problem

How to satisfy a request of size *n* from a list of free holes

- **First-fit**: Allocate the *first* hole that is big enough
- **Best-fit**: Allocate the *smallest* hole that is big enough; must search entire list, unless ordered by size. Produces the smallest leftover hole.
- **Worst-fit**: Allocate the *largest* hole; must also search entire list. Produces the largest leftover hole.

First-fit and best-fit better than worst-fit in terms of speed and storage utilization

❑ **Efficiency Comparison**

We can rank the algorithms based on:

- **Number of processes successfully allocated**
- **Total memory left unused (fragmentation)**

Examples….

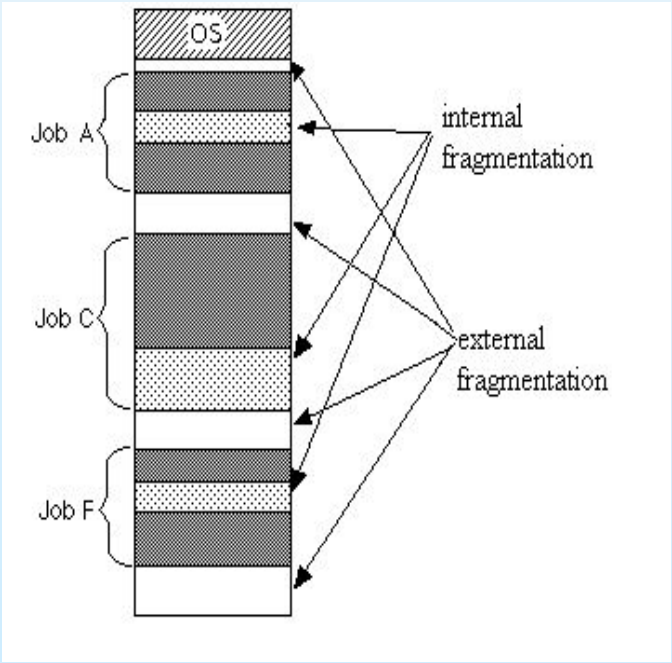
- Consider the requests from processes in given order **300K, 25K**, **125K,** and **50K**. Let there be two blocks of memory available of size **150K** followed by a block size **350K**.

  Which allocation algorithm can satisfy the above requests?

- Given memory partitions of **100K, 500K, 200K, 300K,** and **600K** (in order), how would each of the First-fit, Best-fit, and Worst-fit algorithms place processes of **212K, 417K, 112K,** and **426K** (in order)? Which algorithm makes the most efficient use of memory?

Fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used
- Reduce external fragmentation by **compaction**
  - Shuffle memory contents to place all free memory together in one large block
  - Compaction is possible *only* if relocation is dynamic, and is done at execution time
  - I/O overhead



Paging

- Logical address space of a process can be **non-contiguous**; process is allocated physical memory whenever the latter is available
- Divide physical memory into fixed-sized blocks called **frames** (size is power of 2, between 512 bytes and 8192 bytes)
- Divide logical memory into blocks of same size called **pages**.
- Keep track of all free frames
- To run a program of size *n* pages, need to find *n* free frames and load program
- Set up a page table to translate logical to physical addresses
- **Internal fragmentation**

Paging Calculations

- Consider a system which has logical address which is represented by 7 bits, physical address represented by 6 bits, page size of 8 bytes, then calculate number of pages and number of frames.

  - **Logical address space** =2^7=**128**
  - **Physical address space** =2^6=**64**

  - **Total number of pages** = 2^7= Logical address space/page size=128/8=**16**
  - **Total number of frames** = 2^6= Physical address space/page size=64/8=**8**

- Number of entries in a page table of a process = number of pages in a process =**16**

Address Translation Scheme

- Address generated by CPU is divided into:
  - ***Page number (p)*** – used as an index into a *page table* which contains base address of each page in physical memory
  - ***Page offset (d)*** – combined with base address to define the physical memory address that is sent to the memory unit

Paging Ex.

- **Consider a logical address space of 256 pages with a 4-KB page size, mapped onto a physical memory of 64 frames.**
  1. **How many bits are required in the logical address?**
  2. **How many bits are required in the physical address?**

Logical address space = 256 pages
Page size = 4 KB = 2^{12} bytes
Physical memory = 64 frames

Logical Address Structure:

Each **logical address** consists of:

- **Page number**: Since there are 256 pages → $\log_2(256)=8$ bits
- **Offset within a page**: Page size is 4 KB → $\log_2(4096)=12$ bits

Total logical address size = 8 (page) + 12 (offset) = 20 bits

So, the logical address space is:

- **Size**:=2^{20} =1 MB

Physical Address Structure:

Each **physical address** consists of:

- **Frame number**: 64 frames → $\log_2(64)=6$ bits
- **Offset within frame**: Same as page offset = 12 bits

Total physical address size = 6 (frame) + 12 (offset) = 18 bits

So, the physical address space is:

- **Size**: =2^{18} =256 KB

Address Translation Architecture

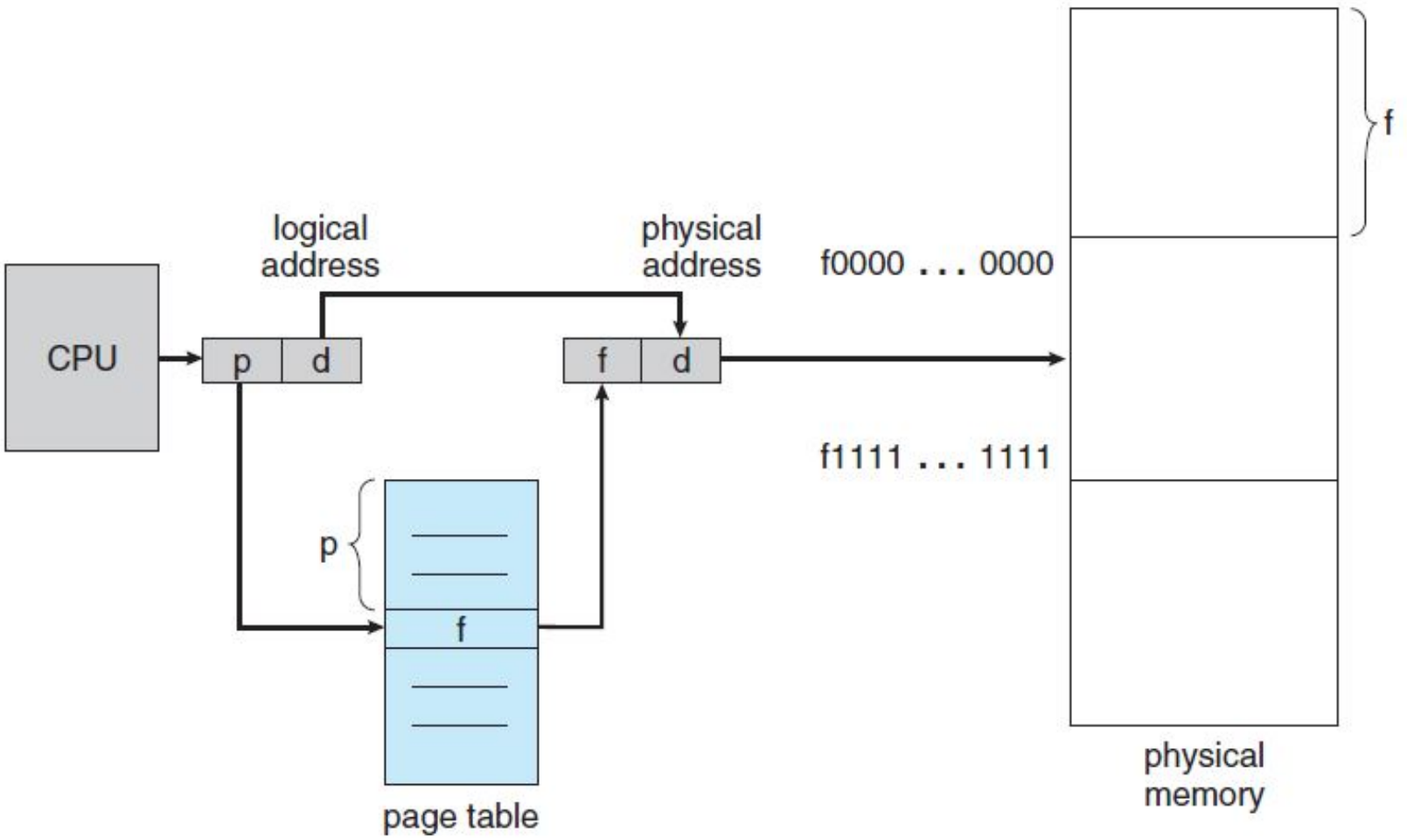


Figure 8.10 Paging hardware.

Paging Example

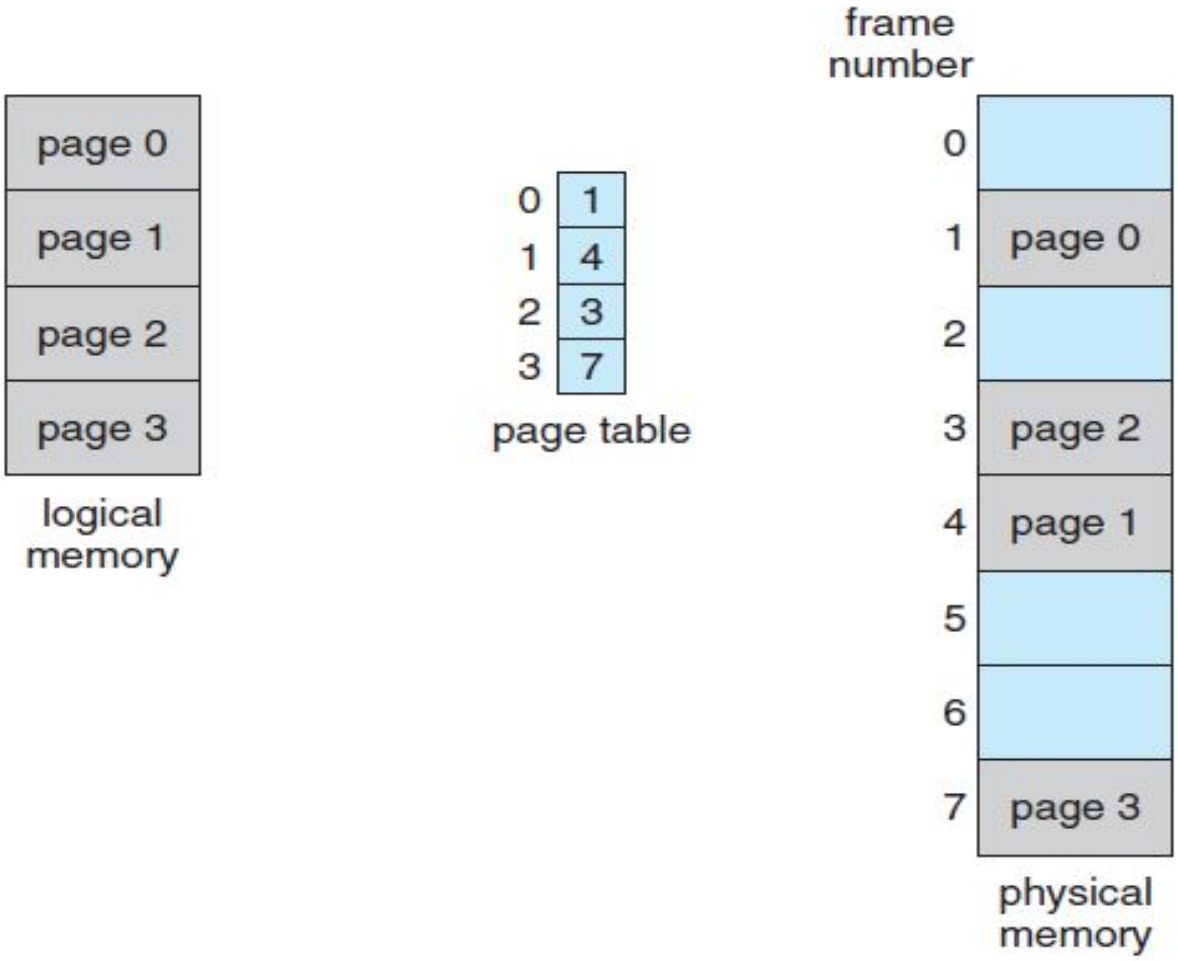


Figure 8.11 Paging model of logical and physical memory.

Paging Example

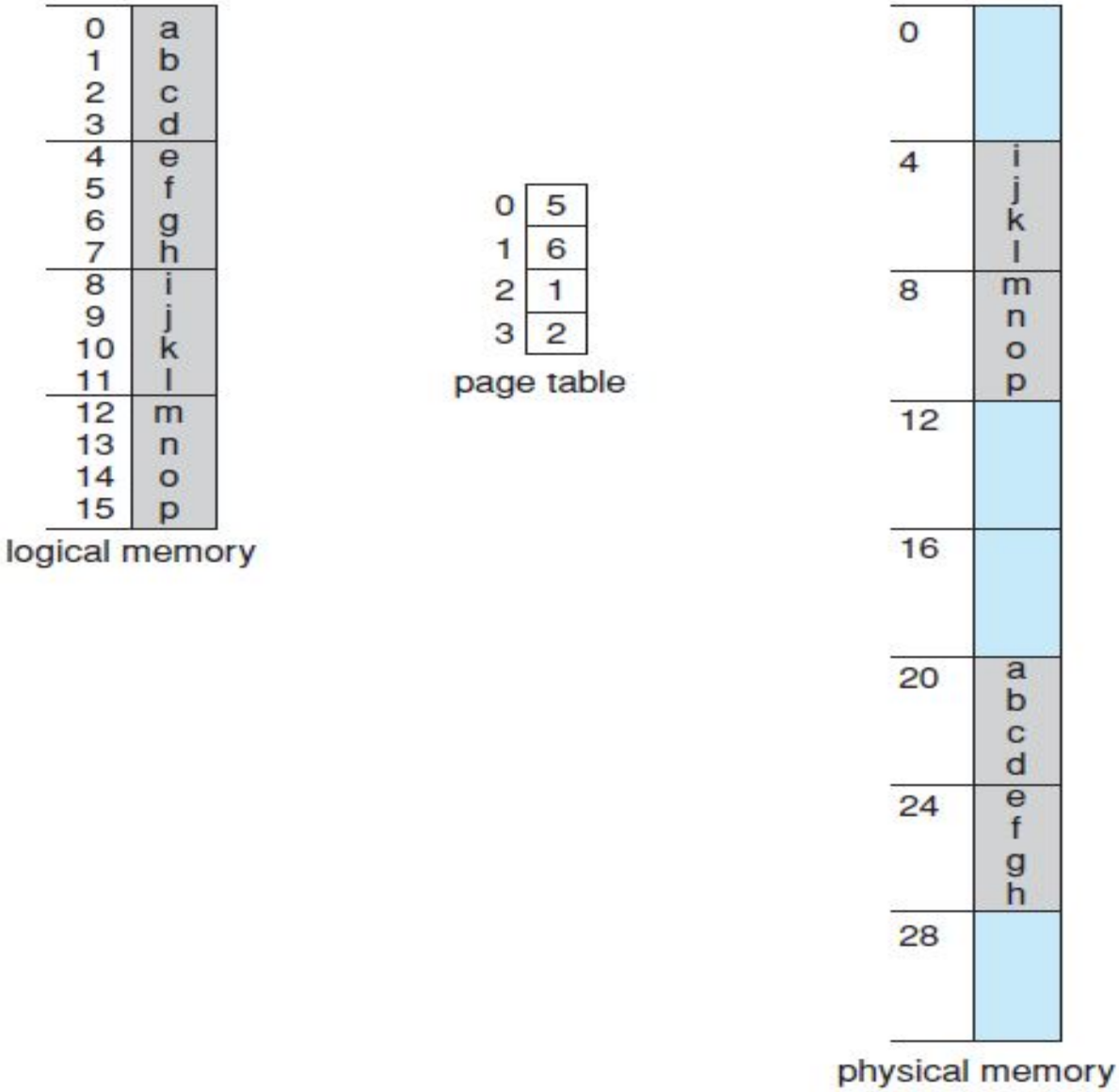


Figure 8.12 Paging example for a 32-byte memory with 4-byte pages.

Free Frames

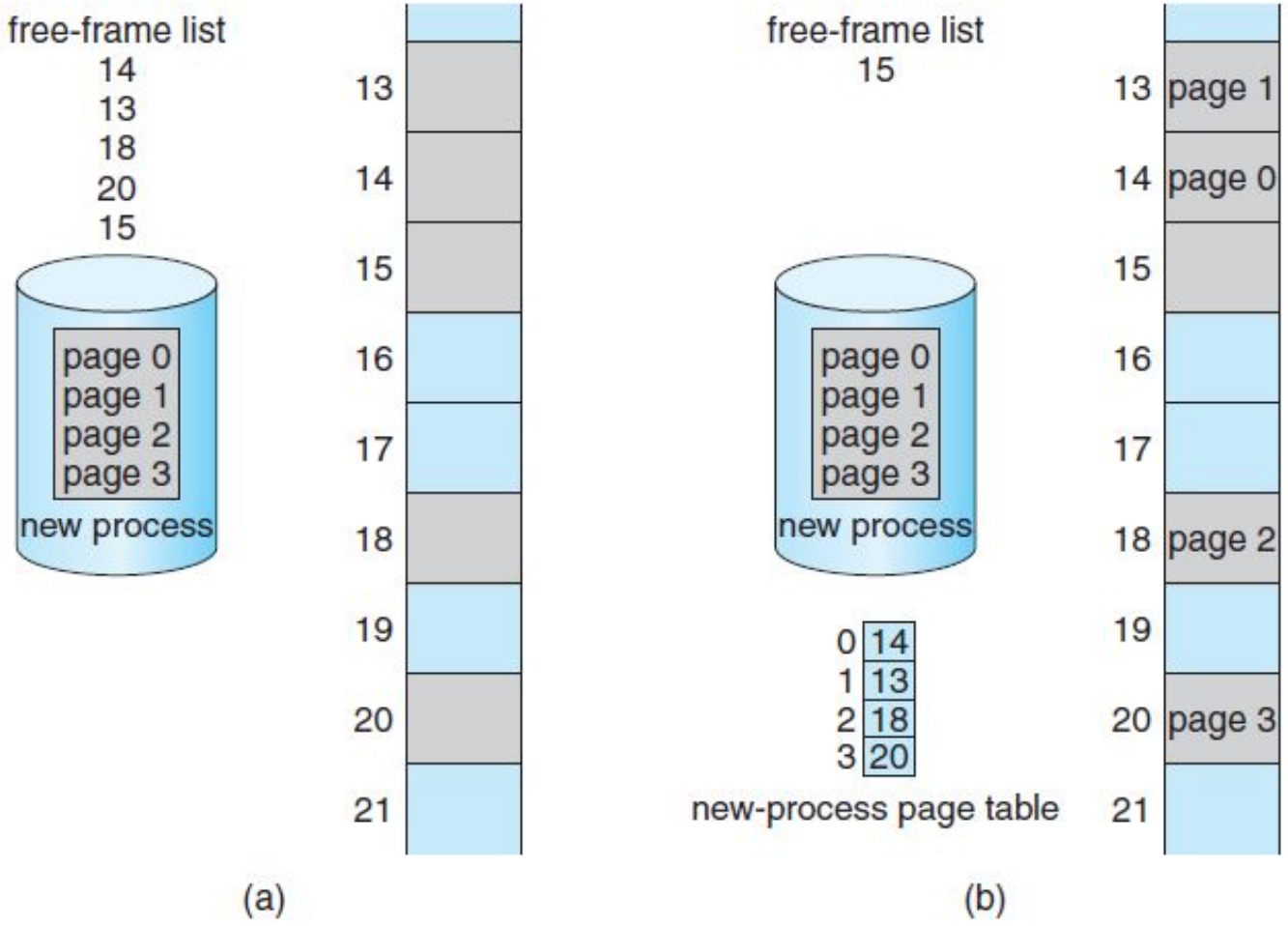


Figure 8.13 Free frames (a) before allocation and (b) after allocation.

Memory Protection

- Memory protection implemented by associating protection bit with each frame

- **Valid-invalid** bit attached to each entry in the page table:
  - "valid" indicates that the associated page is in the process' logical address space, and is thus a legal page
  - "invalid" indicates that the page is not in the process' logical address space

Valid (v) or Invalid (i) Bit In A Page Table

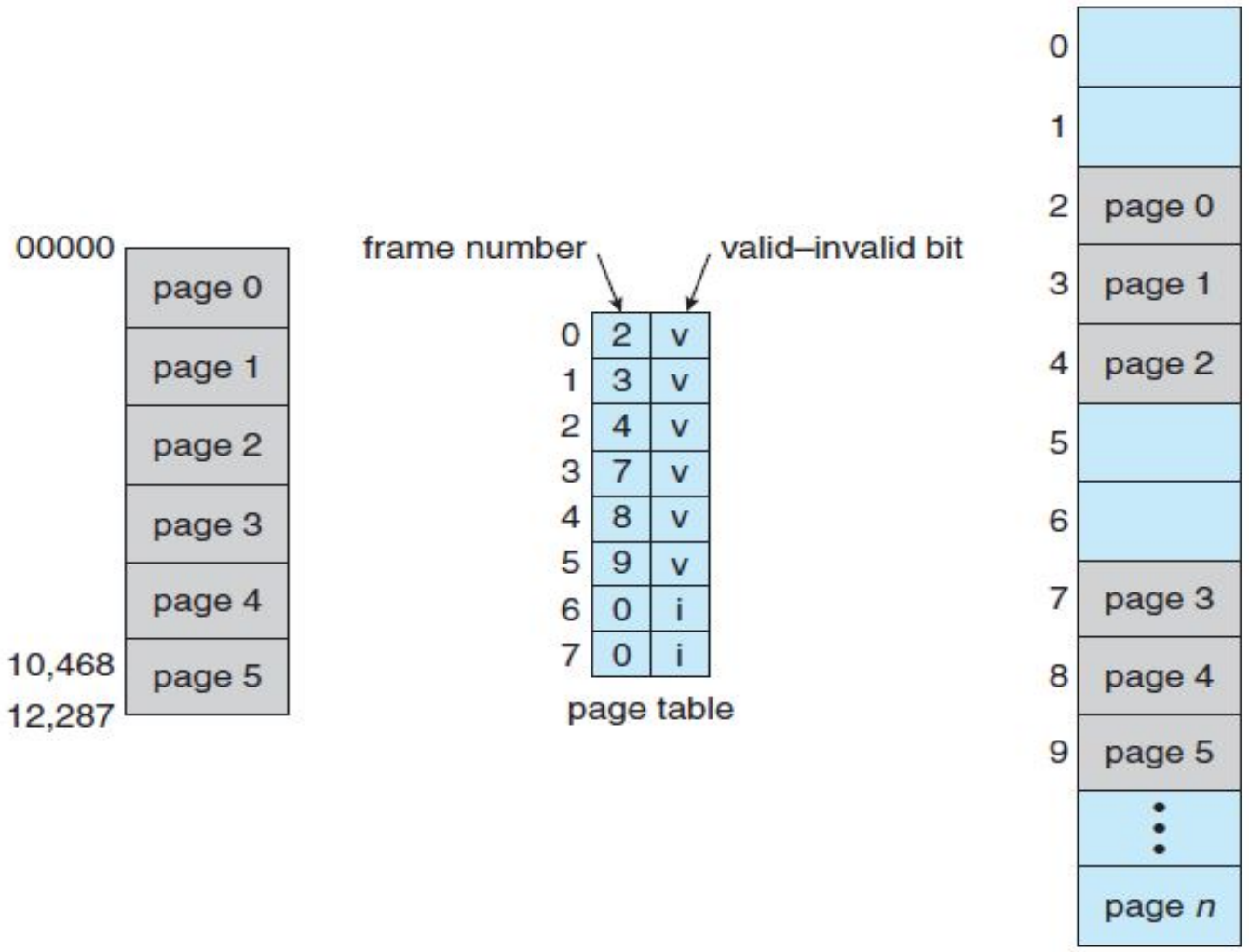


Figure 8.15 Valid (v) or invalid (i) bit in a page table.

Implementation of Page Table

- Page table is kept in main memory
- **Page-table base register** (**PTBR**) points to the page table
- **Page-table length register** (**PTLR**) indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
  - One for the page table and one for the data / instruction
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers** (**TLBs**)

Paging Hardware With TLB

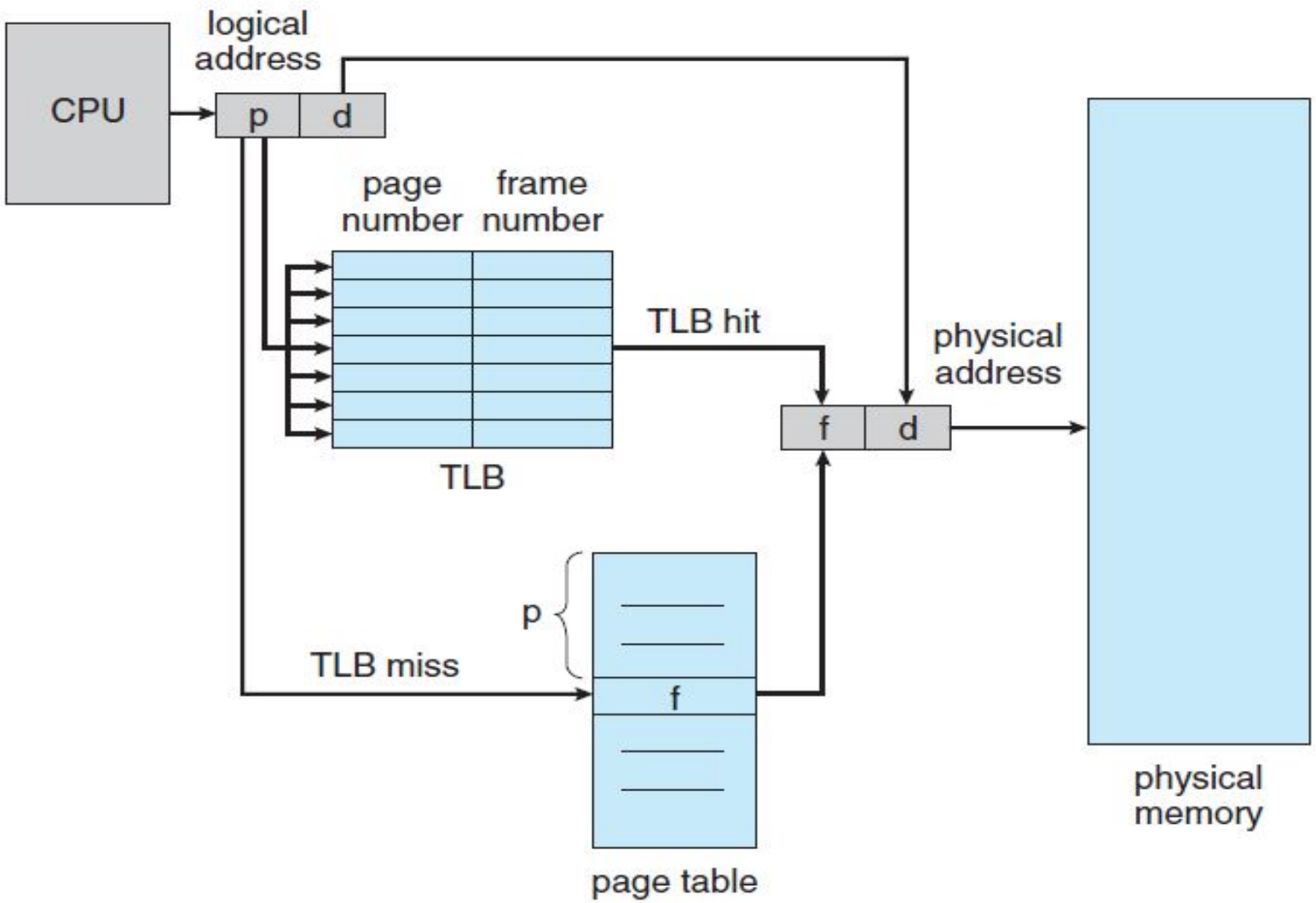
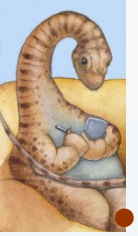


Figure 8.14 Paging hardware with TLB.



Effective Memory Access Time

- The percentage of times that the page number of interest is found in the TLB is called the **hit ratio**. An 80-percent hit ratio, for example, means that we find the desired page number in the TLB 80 percent of the time. If it takes 100 nanoseconds to access memory, then a mapped-memory access takes 100 nanoseconds when the page number is in the TLB. If we fail to find the page number in the TLB then we must first access memory for the page table and frame number (100 nanoseconds) and then access the desired byte in memory (100 nanoseconds), for a total of 200 nanoseconds. (We are assuming that a page-table lookup takes only one memory access, but it can take more, as we shall see.) To find the **effective memory-access time**, we weight the case by its probability:

  effective access time = 0.80 × 100 + 0.20 × 200 = 120 nanoseconds

- In this example, we suffer a 20-percent slowdown in average memory-access time (from 100 to 120 nanoseconds).For a 99-percent hit ratio, which is much more realistic, we have

  effective access time = 0.99 × 100 + 0.01 × 200 = 101 nanoseconds

Shared Pages

- **Shared code**
  - One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems).
  - Shared code must appear in same location in the logical address space of all processes

- **Private code and data**
  - Each process keeps a separate copy of the code and data
  - The pages for the private code and data can appear anywhere in the logical address space

Shared Pages Example

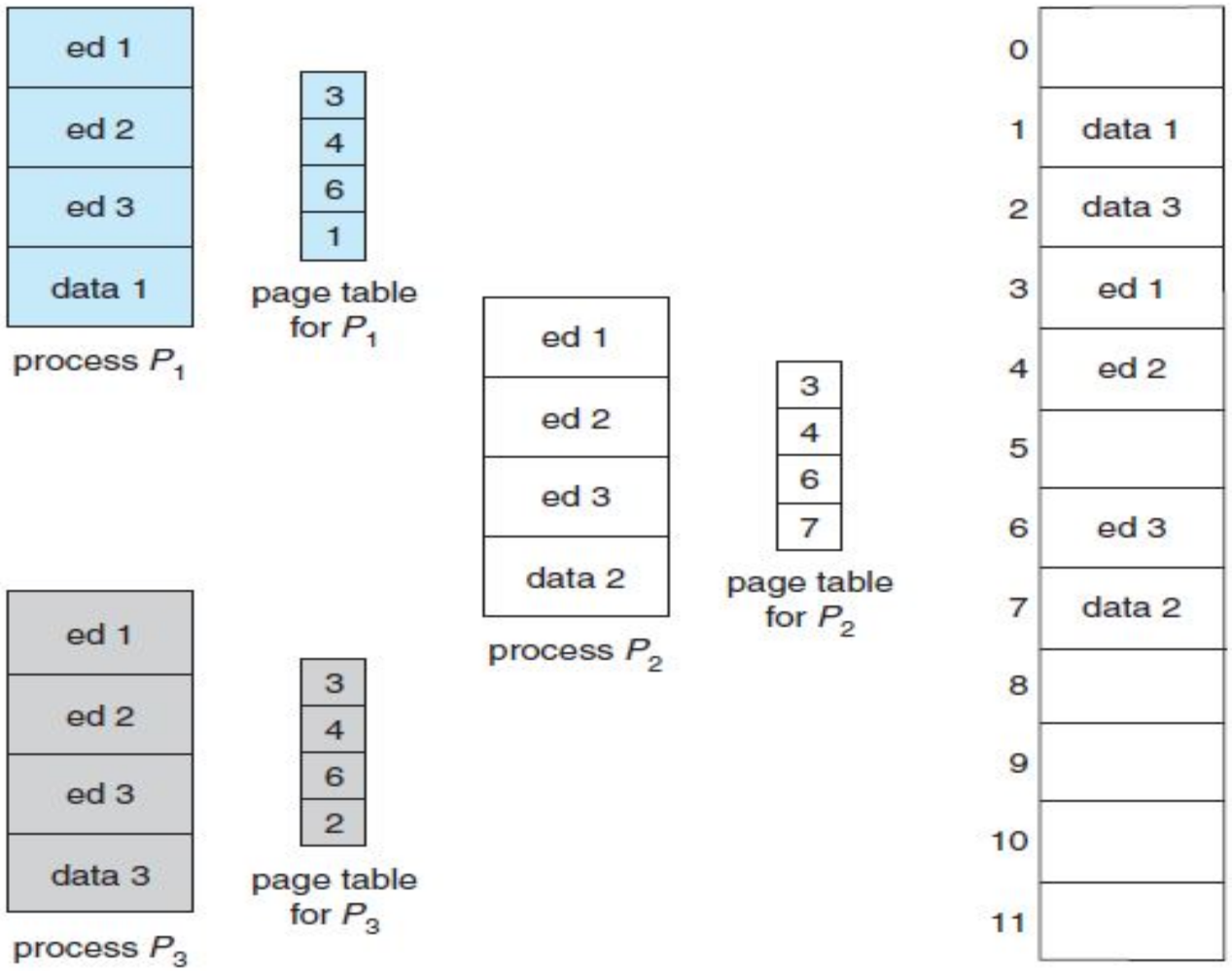


Figure 8.16 Sharing of code in a paging environment.

Segmentation

- Memory-management scheme that supports user view of memory
- A program is a collection of segments. A segment is a logical unit such as:
  main program,
  procedure,
  function,
  method,
  object,
  local variables, global variables,
  common block,
  stack,
  symbol table, arrays

User's View of a Program

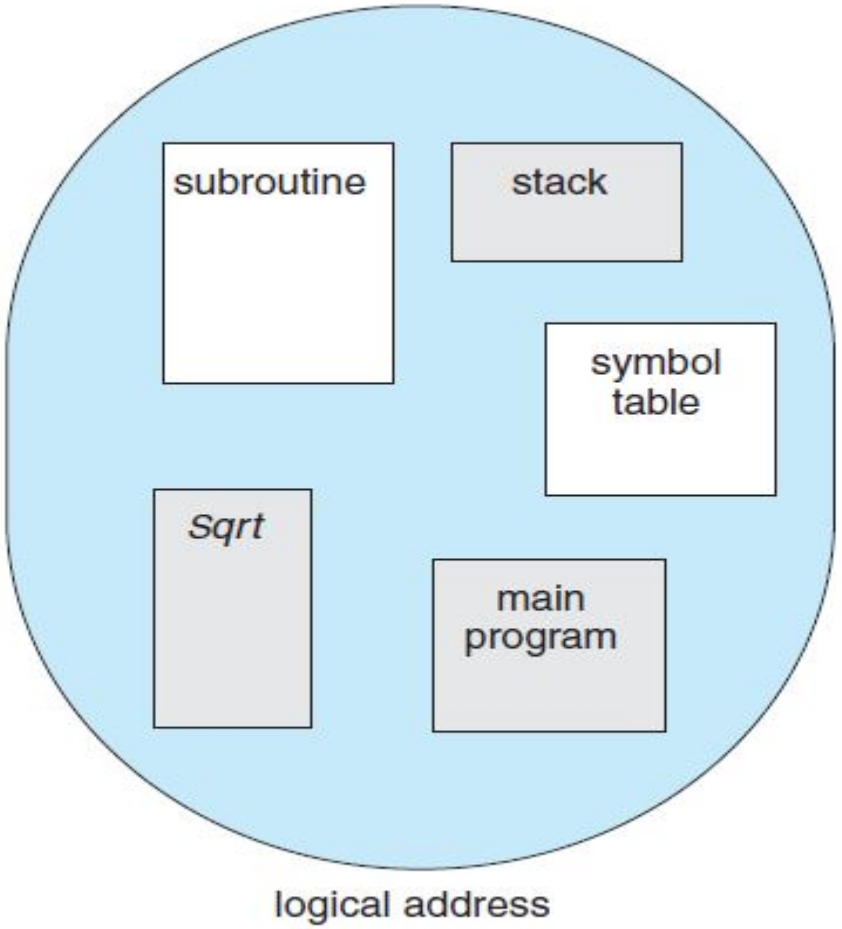
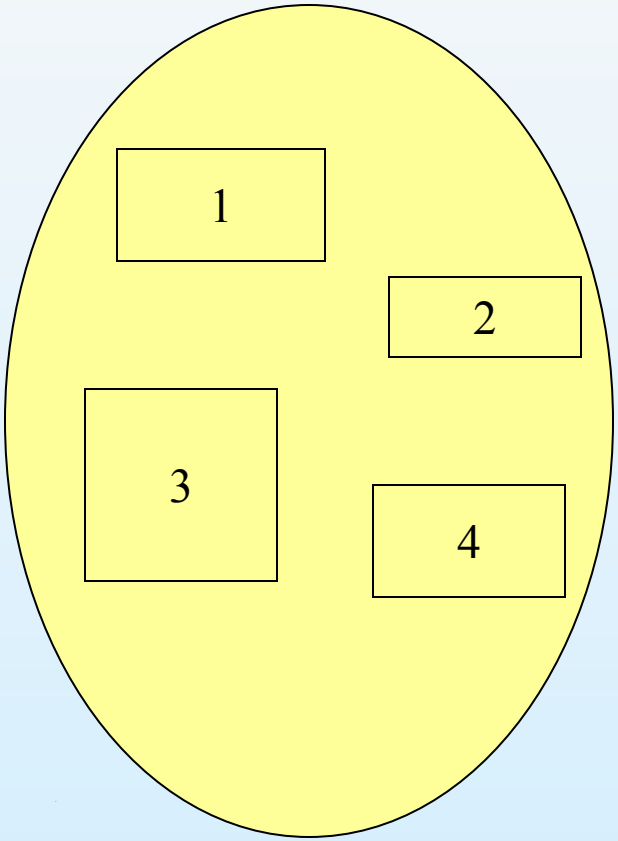
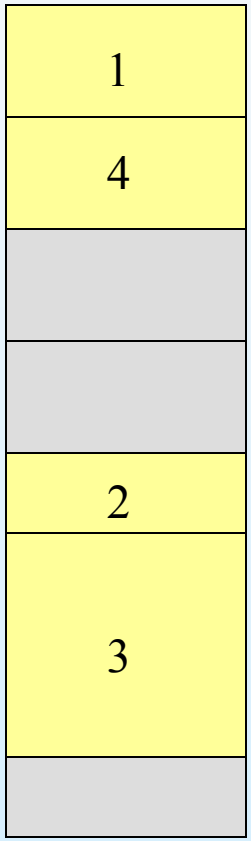


Figure 8.7 Programmer's view of a program.

Logical View of Segmentation



user space



physical memory space

Segmentation Architecture

- Logical address consists of a two tuple:

  <segment-number, offset>,

- **Segment table** – maps two-dimensional physical addresses; each table entry has:
  - base – contains the starting physical address where the segments reside in memory
  - *limit* – specifies the length of the segment
- *Segment-table base register (STBR)* points to the segment table's location in memory
- *Segment-table length register (STLR)* indicates number of segments used by a program;

  segment number s is legal if $s <$ STLR

Address Translation Architecture

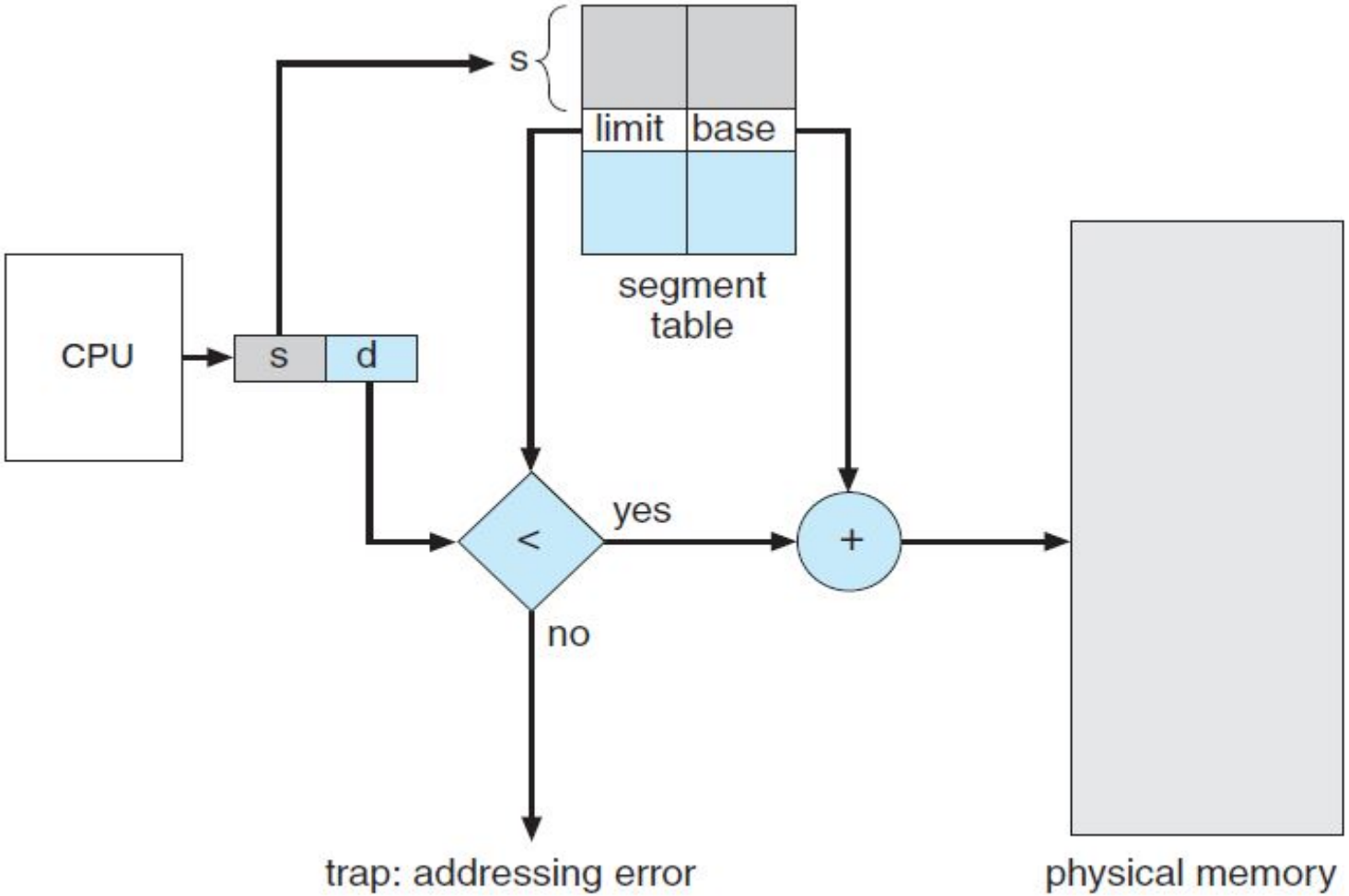


Figure 8.8 Segmentation hardware.

Example of Segmentation

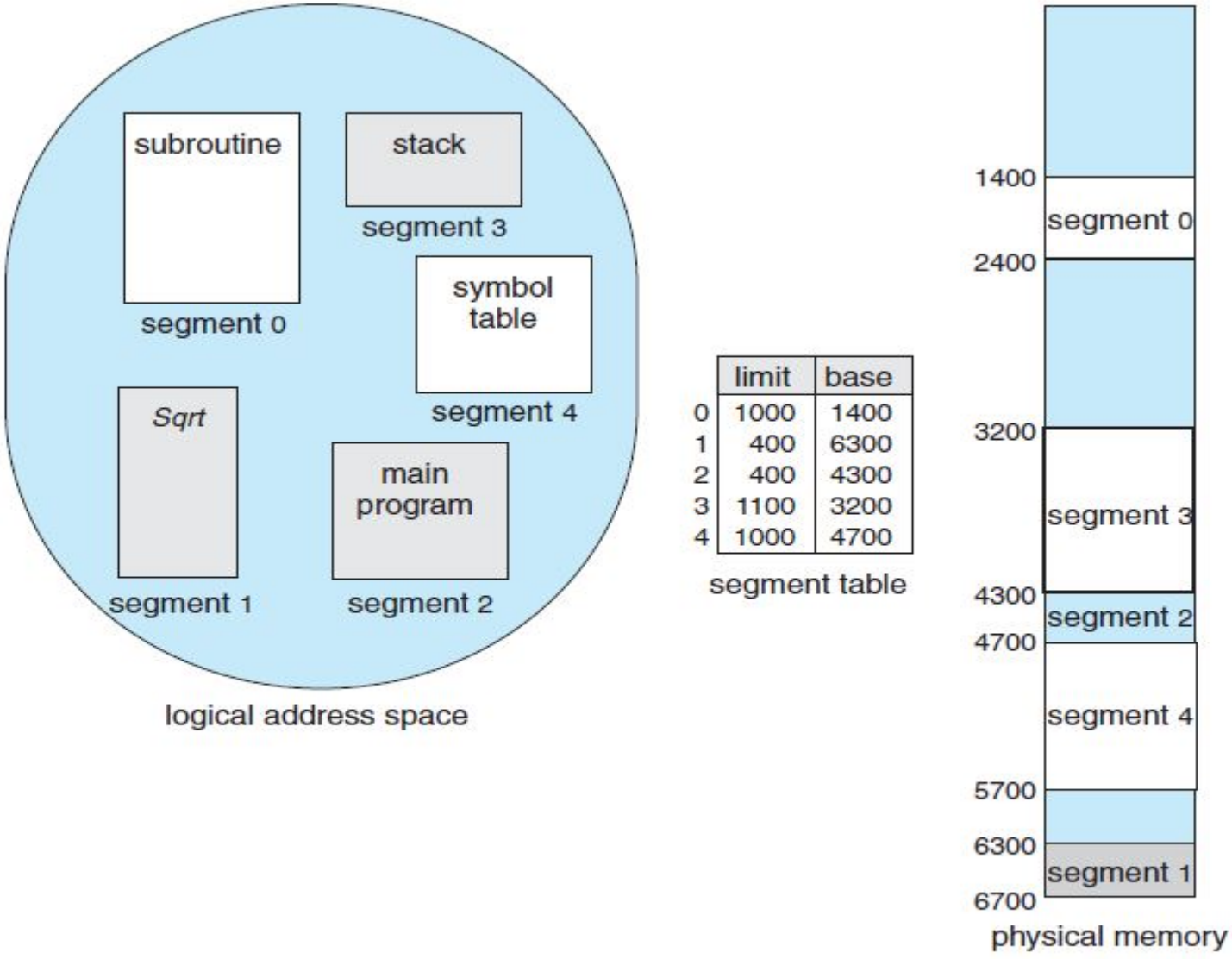
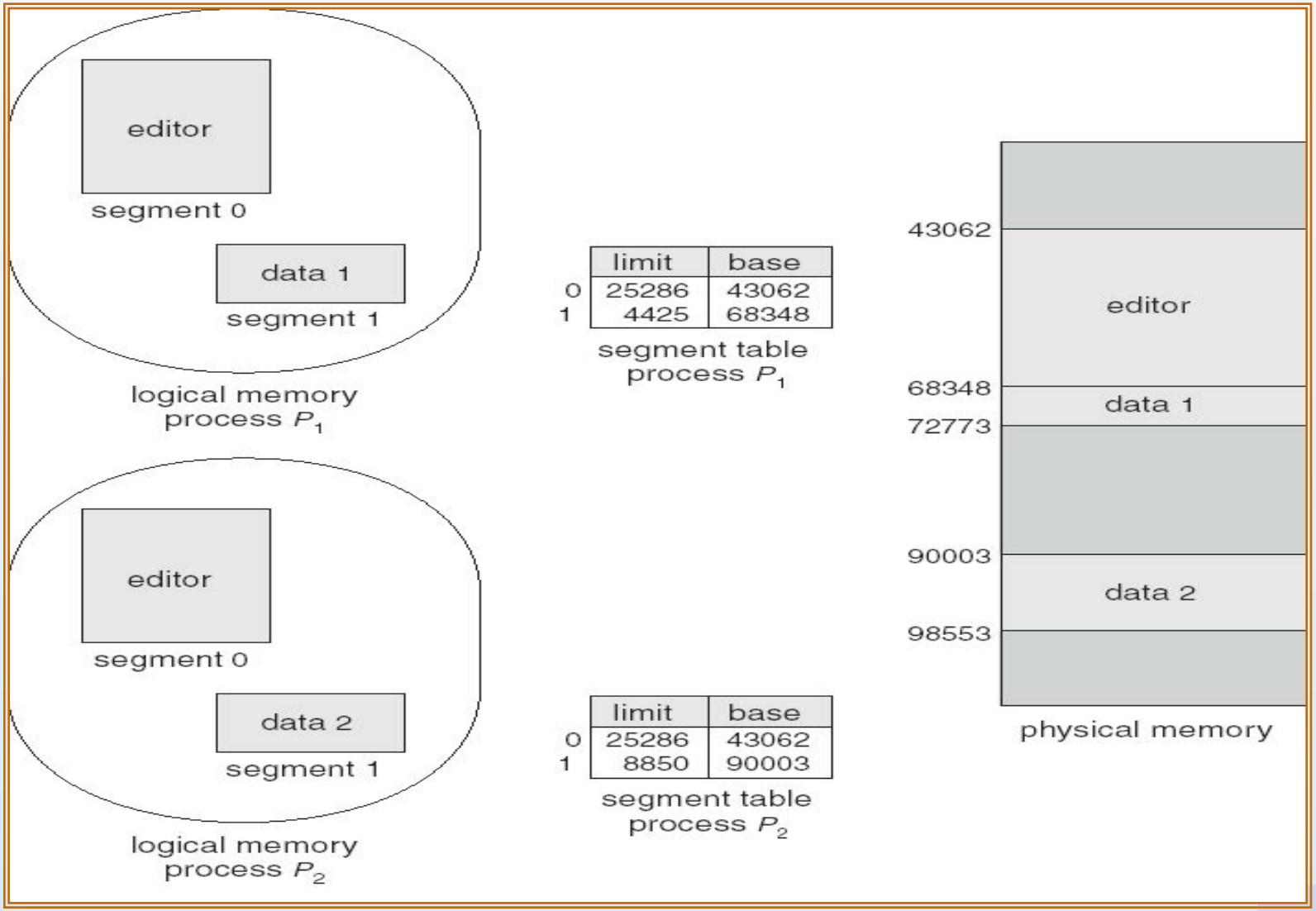


Figure 8.9 Example of segmentation.

Sharing of Segments



Segmentation Architecture (Cont.)

- Protection. With each entry in segment table associate:
  - validation bit = 0 ⇒ illegal segment
  - read/write/execute privileges
- Since segments vary in length, memory allocation is a **dynamic storage-allocation problem**
- Segmentation leads to **external fragmentation**
- Code sharing occurs at segment level; shared segments

End of Chapter 8